

csiro-biomass

3位Solution

Rank	3
Team	embee
Author	Mayukh Bhattacharyya
Topic ID	670901
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/csiro-biomass/discussion/670901>

Retrieved with Kaggle CLI on 2026-07-18.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

このコンペを開催してくださったCSIROとKaggle、そして期間中にアイデアを共有してくださった皆さんに感謝します。

これほど上位になるとは正直驚きました。shake-upへの保険としてheavy ensembleに賭けましたが、それが順位を上げる助けにもなるとは思っていませんでした。私の提出は、model typeとCV strategyの異なる19 modelのensembleです。

1. Model architecture

2種類のmodel familyを学習しました。

- Metadata model
- Metadataを使わないmodel

Metadata model

最初は vit_huge_plus_patch16_dinov3 backboneを使い、全metadataを予測するauxiliary headを持つmodelを学習しました。その予測を最終headへ入力してbiomassを予測します。main lossにauxiliary metadata lossを加えて学習しました。その後、metadataは実際にはほとんど差を生まないと分かったため、使うのをやめました。

Metadataを使わないmodel

皆が使っていた人気のMambaBlock modelを採用し、まず2 backbone、最終的には3 backboneで動かしました。1つ目は、互いをよく補完する vit_huge_plus_patch16_dinov3 と convnextv2_large です。2つ目ではさらに vit_base_patch16_224.mae を加えました。私の経験ではmodel選択によるperformance差は非常に小さく、どのmodelも4/5-fold ensembleで0.64以上でした。ただし、私の場合はDINOv3 Huge PlusがLargeを常に上回りました。

1 modelを除き、すべて左右へ画像を分割する2-stream modelです。例外として画像全体を入力するmodelも試しましたが、performanceはほぼ同じでした。

すべてのmodelは3 headを持ち、GDMとTotalは3つの予測値から導出しました。

学習時はpre-trained weightを完全に壊さないよう、backboneのlearning rateを通常の1/10にしました。

```
class BiomassModelV2(nn.Module):
    """DINOv3 + Mamba Fusion + Multi-Head Regression"""

    def __init__(self, model1_name: str,
                  model2_name: str,
                  pretrained: bool = True
                  ):
        super().__init__()
        self.model1_name = model1_name
        self.model2_name = model2_name

        self.backbone1 = timm.create_model(
            model1_name, pretrained=pretrained, num_classes=0, global_pool=''
```

```

    )
    self.backbone2 = timm.create_model(
        model2_name, pretrained=pretrained, num_classes=0, global_pool=''
    )
    nf1 = self.backbone1.num_features
    nf2 = 256
    nf = nf1 + nf2

    self.fusion1 = nn.Sequential(
        LocalMambaBlock(nf1, kernel_size=5, dropout=0.1),
        LocalMambaBlock(nf1, kernel_size=5, dropout=0.1)
    )
    self.fusion2 = nn.Sequential(
        LocalMambaBlock(nf2, kernel_size=5, dropout=0.1),
        LocalMambaBlock(nf2, kernel_size=5, dropout=0.1)
    )
    self.pool = nn.AdaptiveAvgPool1d(1)

    self.head_green = nn.Sequential(
        nn.Linear(2*nf, nf // 2), nn.GELU(), nn.Dropout(0.2),
        nn.Linear(nf // 2, 1), nn.Softplus()
    )
    self.head_dead = nn.Sequential(
        nn.Linear(2*nf, nf // 2), nn.GELU(), nn.Dropout(0.2),
        nn.Linear(nf // 2, 1), nn.Softplus()
    )
    self.head_clover = nn.Sequential(
        nn.Linear(2*nf, nf // 2), nn.GELU(), nn.Dropout(0.2),
        nn.Linear(nf // 2, 1), nn.Softplus()
    )
    )

def forward(self, x):
    # x is a tuple (left, right)
    left, right = x

    x_l1 = self.fusion1(self.backbone1(left))
    x_l2 = self.fusion2(self.backbone2(left).flatten(2))
    x_r1 = self.fusion1(self.backbone1(right))
    x_r2 = self.fusion2(self.backbone2(right).flatten(2))

    x_l1 = self.pool(x_l1.transpose(1, 2)).flatten(1)
    x_l2 = self.pool(x_l2.transpose(1, 2)).flatten(1)
    x_r1 = self.pool(x_r1.transpose(1, 2)).flatten(1)
    x_r2 = self.pool(x_r2.transpose(1, 2)).flatten(1)

    x_cat = torch.cat([x_l1, x_l2, x_r1, x_r2], dim=1)

    green = self.head_green(x_cat)
    dead = self.head_dead(x_cat)
    clover = self.head_clover(x_cat)
    gdm = green + clover
    total = gdm + dead

    # Return as a single tensor (batch, 5)
    return torch.cat([green, dead, clover, gdm, total], dim=1)

```

Data

このコンペで最大の要因がdataだったことは明らかで、当初から提供された訓練データから最大限の情報を引き出すことに注力しました。主要なaugmentationはすべて使いました。modelが色だけでなくfeatureを学ぶよう、特にrandom grayscaleを使いました。

効果があったと思うaugmentationを2つ考案しました。

- lambda 0.2~0.3の単純なMixUp。
- 画像に縦方向のrandom cutを3本入れて4 segmentへ分割し、segment順を入れ替える方法。並べ替えても画像内のbiomass量は変わらないため、有効なaugmentationです。同じ考えで、左右画像も50%の確率で入れ替えました。

この2つは片方だけでなく、同時に適用しました。

CV

2種類のCV、正確にはfold作成法を試しました。最初の1か月以降はCV scoreを諦め、LB scoreだけを信頼しました。overfittingを避けるため、あるarchitectureを学習するときは、その全fold modelを常に使うことにしました。どちらのCV strategyもrobustには感じられず、LBで良い1~2 foldと悪い1~2 foldが必ずありました。

- 1つ目はdateをgroupにしたGroupKFoldです。
- 2つ目はやや複雑で、公開されたembedding distance CVが基礎です。各validation setをnearest neighbors距離に基づくeasy/hardの同数subsetへ拡張しました。当初はhard subsetで高scoreのweightほどLBに整合すると予想しましたが、実際は正反対でした。easy subsetの方がLBと一貫してよく整合しました。testのhard sampleは本当に難しく、easy/hard双方で「良い」modelより、easyで「非常に良く」、hardでは「まずまず」のmodelの方が高scoreを取れるのだと思います。

```
def create_robust_cv(
    df,
    embeddings_col,
    target_names,
    n_splits=5,
    n_clusters=50,
    seed=42,
    k=5,
    distance_metric="euclidean",
):
    df = df.copy().reset_index(drop=True)

    # 1. Embeddings
    X = np.vstack(df[embeddings_col].values).astype(np.float32)
    X /= np.linalg.norm(X, axis=1, keepdims=True) + 1e-8

    # 2. Visual clustering
    kmeans = KMeans(
        n_clusters=n_clusters,
        random_state=seed,
        n_init=10,
    )
    df["visual_cluster"] = kmeans.fit_predict(X)

    # 3. Stratification target
```

```

weights = {
    "Dry_Green_g": 0.1,
    "Dry_Dead_g": 0.1,
    "Dry_Clover_g": 0.1,
    "GDM_g": 0.2,
    "Dry_Total_g": 0.5,
}

composite = sum(df[t] * weights.get(t, 0) for t in target_names)
try:
    df["target_bins"] = pd.qcut(composite, q=10, labels=False, duplicates="drop")
except ValueError:
    df["target_bins"] = pd.qcut(composite, q=5, labels=False, duplicates="drop")

# 4. CV split
sgkf = StratifiedGroupKFold(
    n_splits=n_splits,
    shuffle=True,
    random_state=seed,
)
df["fold"] = -1
df["val_difficulty"] = None
df["mean_knn_dist_to_train"] = np.nan

for f, (train_idx, val_idx) in enumerate(
    sgkf.split(df, df["target_bins"], groups=df["visual_cluster"])
):
    X_train, X_val = X[train_idx], X[val_idx]

    nn = NearestNeighbors(n_neighbors=k, metric=distance_metric)
    nn.fit(X_train)
    knn_dist, _ = nn.kneighbors(X_val)
    mean_dist = knn_dist.mean(axis=1)
    df.loc[val_idx, "mean_knn_dist_to_train"] = mean_dist

# 5. EASY / HARD split
order = np.argsort(mean_dist)
mid = len(order) // 2

easy = val_idx[order[:mid]]
hard = val_idx[order[mid:]]

df.loc[easy, ["fold", "val_difficulty"]] = [f, "easy"]
df.loc[hard, ["fold", "val_difficulty"]] = [f + n_splits, "hard"]
return df

```

Inference

- TTAはLB performanceを悪化させたため無効にしました。heavy augmentationとensembleを使っていたので、最終的には不要でした。
- 公開した[notebook](#)のscalingを使い、少し拡張しました。Cloverはtestでdata分布が大きく異なるようでした。Dry Deadではmodelが平均へ保守的に予測する一方、実値は両端寄りだと感じました。scalingはPublic LBを0.01未満改善しました。

CVもLBも信頼できなかったため、当初から複数architectureを学習してensembleする方針でした。modelやtrainingへ大きな変更を加えず、CV scoreの安定性とLB改善に基づいて、ゆっくり段階的に改良しました。

個々のarchitectureは5-foldでほぼ同等（約0.64以上）でした。最後に学習した3-backbone・metadataなしのmodel setが最高scoreの提出で、0.67でした。

最後に、synthetic datasetとtest-time trainingを成功させようとしたのですが、特にtest dataが大きく異なったため失敗しました。上位teamがそれを成功させた方法を読むのは、とても興味深いです。